

Milestone 1: Production RAG System

DATA 790 - Advanced AI Frameworks in Production

William McDonald | September 30, 2026

Purpose. This milestone implements and evaluates a compact production-style Retrieval-Augmented Generation (RAG) system over a course-focused corpus. The system retrieves relevant chunks, generates grounded answers, returns source identifiers, measures retrieval and answer quality, records latency and token usage, and projects operating cost. The goal is a defensible baseline that can be hardened and deployed in later milestones.

1. Architecture and Design Rationale

The system is intentionally simple enough to inspect end to end while still separating the major production RAG responsibilities: ingestion, chunking, embedding, vector storage, retrieval, generation, attribution, validation, evaluation, and cost tracking.

Course corpus -> Chunking comparison -> Embeddings -> Chroma vector store
 Question -> Input validation -> Top-3 retrieval -> Retrieved context -> gpt-4o-mini
 Answer + source chunk IDs -> Optional support check -> Evaluation / latency / cost

Component	Implementation	Rationale
Corpus / ingestion	10 structured course sections with stable section/title metadata	Keeps the Milestone 1 corpus controlled and makes source attribution and section-level evaluation explicit.
Chunking	180-character chunks with 30-character overlap	Selected empirically from three tested configurations because it preserved perfect top-3 coverage while improving precision.
Embeddings	text-embedding-3-small via UNC AI Gateway	Provides semantic vector representations through the course-supported API at low operational complexity.
Vector store	Chroma with cosine similarity	Matched the small local workload, achieved perfect Recall@10 in the Lab 5 benchmark, and had the lowest measured local query latency.
Retrieval	Top-3 chunks	Balances context coverage against prompt size; Hit Rate@3 was 1.000 on all 22 evaluation questions.
Generation	gpt-4o-mini, temperature 0	A smaller model is sufficient for grounded question answering and supports low latency and low projected cost.
Attribution	Chunk ID and section returned with each answer	Makes retrieved evidence visible to the user and supports debugging and evaluation.
Guardrail / support	Input pattern checks plus optional support judge and retry at k=5	Provides a baseline safety and grounding layer while leaving stronger controls for Milestone 2.

Implementation quality and extensibility

The notebook separates major responsibilities into reusable functions, including `chunk_text()`, `build_chunks()`, `embed_texts()`, `validate_input()`, `retrieve()`, `build_context()`, `generate_answer()`, `ask_question()`, `concept_score()`, and `judge_answer()`. The API key is loaded from the environment rather than embedded in code, missing credentials fail early, and invalid or suspicious user inputs are blocked before retrieval. The architecture is extensible because the vector store, retrieval depth, generation model, support check, and evaluation logic are isolated rather than hard-coded into one monolithic pipeline.

This is still a Milestone 1 baseline rather than a fully hardened service. API retry/backoff, persistent vector storage, richer authorization, and production monitoring are intentionally deferred and are listed as Milestone 2 work rather than presented as completed features.

2. Chunking Strategy and Vector Store Evidence

Three fixed-window chunking configurations were evaluated against the same 22-question golden set. For each strategy, the corpus was re-chunked, re-embedded, loaded into a fresh Chroma collection, and queried with $k=3$. The selected strategy maximized retrieval coverage first and precision second.

Strategy	Chunk size / overlap	Chunks	Precision@3	Recall@3	Hit Rate@3
Small	180 / 30	21	0.485	1.000	1.000
Medium	320 / 60	11	0.333	1.000	1.000
Large	600 / 100	10	0.333	1.000	1.000

All three strategies retrieved the expected section within the top three results for every test question. The small strategy was selected because its Precision@3 was higher (0.485 versus 0.333) while Recall@3 and Hit Rate@3 remained perfect. This result is consistent with the corpus: the source sections are short and concept-dense, so smaller chunks reduce unrelated text without losing section coverage. The tradeoff is that character boundaries can split a sentence or term, which appears in one failure case below.

The final vector-store choice also reused evidence from the Lab 5 benchmark. Chroma achieved Recall@10 of 1.000 with approximately 1.51 ms query latency in the local test. Pinecone also reached 1.000 recall but with 120.85 ms measured latency, while Weaviate reached 0.933 recall at 54.05 ms. These latency values are not directly equivalent because Chroma was local while Pinecone and Weaviate were cloud services; the comparison is therefore used as practical evidence for this small project, not as a universal database ranking.

Database	Insert	Query latency	Recall@10
Chroma	0.009 s	1.51 ms	1.000
Pinecone	0.70 s	120.85 ms	1.000
Weaviate	1.15 s	54.05 ms	0.933

3. Retrieval and Answer Evaluation

The final evaluation set contains 22 questions: 10 written manually, 10 AI-assisted questions verified against the corpus, and 2 edge cases. This exceeds the approximately 20-question baseline while keeping every question tied to a known expected section and one or more expected answer concepts.

Metric definitions are intentionally simple and reproducible. Precision@3 is the fraction of the three returned chunks that belong to the expected section. Recall@3 is 1 when at least one expected-section chunk is present in the top three and 0 otherwise. Hit Rate@3 uses the same binary criterion in this baseline; because each question maps to one expected section, Recall@3 and Hit Rate@3 are equivalent by construction. Answer quality is measured both with an exact concept check and an LLM judge scored as 0, 0.5, or 1.

Metric	Final run	Interpretation
Precision@3	0.485	On average, 1.45 of the top 3 chunks came from the expected section.
Recall@3	1.000	Every question had at least one expected-section chunk in the top 3.
Hit Rate@3	1.000	Expected-section retrieval succeeded on all 22 questions.
Average concept score	0.909	Most generated answers contained the expected answer concepts.
Average LLM judge score	0.841	Answers were generally correct, with two weak cases identified for analysis.
Average retrieval latency	5.42 ms	Vector retrieval was a small fraction of total response time.
Average generation latency	869.27 ms	Model generation dominated end-to-end latency.

Failure analysis

Failure 1 - "What are the main stages in a RAG pipeline?" The retriever still located the expected RAG overview section, so this was primarily a generation/context-window failure rather than a retrieval miss. The generated answer was "I do not know based on the provided corpus," producing both a concept score of 0 and an LLM judge score of 0. The likely cause is the selected 180-character chunk boundary: the list of pipeline stages is distributed across adjacent text, so an individual top-3 context set may not present the complete sequence cleanly. The first knob I would turn is chunk size or retrieval depth for list-style questions; a sentence-aware chunker would be a later improvement.

Failure 2 - "Name one way to reduce RAG cost." The returned answer was related to cost optimization but did not include the expected concept "cache." The exact concept scorer therefore assigned 0, while the LLM judge assigned 0.5, indicating a partially useful but incomplete answer. This exposes two issues: small chunks can omit the specific optimization term even when the correct section is retrieved, and exact-match evaluation can be harsher than semantic correctness. Retaining both the deterministic concept score and the LLM judge gives a more defensible view of answer quality.

4. Cost Analysis and Production Projection

The notebook tracks prompt and completion token usage from the primary generation call and applies the same gpt-4o-mini pricing snapshot used in Lab 3. The final evaluation averaged 189.45 prompt tokens and 16.77 completion tokens per query. At the projected volume of 10,000 queries per month, the estimated primary inference cost is approximately \$0.38 per month.

Item	Estimate
Average prompt tokens / query	189.45
Average completion tokens / query	16.77
Estimated generation cost / query	\$0.0000385
Projected volume	10,000 queries / month
Projected monthly generation cost	\$0.3848
Optimization lever	Cache repeated questions
Illustrative cache hit rate	20%
Estimated monthly savings	\$0.0770
Projected cost after cache	\$0.3079

Caching is the clearest first optimization lever for this workload because repeated questions can avoid another generation call entirely. The 20% example is illustrative rather than a measured production hit rate. This projection covers the primary answer-generation call only; embedding requests, LLM-as-judge evaluation calls, optional support-check calls, storage, and infrastructure overhead are outside this simple Milestone 1 estimate.

5. Limitations and Milestone 2 Priorities

The current system demonstrates the complete RAG loop but intentionally leaves several production concerns for the next milestone. The most important limitations are:

- Character-based chunking can split words, sentences, or lists; a sentence- or token-aware strategy should be tested against the same golden set.
- Chroma is ephemeral and local in the notebook; production use would require persistent storage, backup, operational monitoring, retries, rate-limit handling, and structured error logging.
- Prompt-injection protection is rule-based pattern matching. Milestone 2 should add stronger classification, output filtering, access control, logging, and auditability.
- The evaluation set is intentionally small and course-specific. A broader production test set should include more multi-hop, numeric, negation, and unanswerable questions.

Conclusion

Milestone 1 establishes a measurable RAG baseline. Across 22 golden questions, retrieval achieved a 1.000 Hit Rate@3 and the answer pipeline achieved a 0.841 average LLM judge score while returning source chunk IDs and measuring latency, token usage, and projected cost. The documented failure cases define the next work for Milestone 2: security hardening, persistence, monitoring, and more robust evaluation.